
Taller de Comparación de Paradigmas de Paralelismo

Multiplicación de Matrices: Implementaciones Paralelas

1. Descripción general

Este taller tiene como objetivo comparar distintos modelos y frameworks de programación paralela a través de un problema de referencia común: la multiplicación de matrices cuadradas de punto flotante de precisión simple (fp32). Cada grupo de trabajo implementa, analiza y expone una tecnología de paralelización diferente, discutiendo su modelo de programación, su portabilidad, sus restricciones de hardware y su rendimiento medido frente a las implementaciones de referencia.

Problema objetivo: $C = A \times B$, donde A, B y C son matrices cuadradas de dimensión $N \times N$ almacenadas en formato row-major como arreglos 1D de `float`.

Tamaños a evaluar: $N \in \{512, 1024, 2048, 4096\}$

Métrica principal: Tiempo de ejecución del kernel/región paralela en milisegundos y GFLOPS efectivos.

Referencia de GFLOPS: La multiplicación $N \times N$ requiere exactamente $2N^3$ operaciones en punto flotante (N^2 productos y $N^2 - 1 \approx N^2$ sumas por elemento).

2. Objetivos

2.1 Objetivo general

Comparar el modelo de programación, la portabilidad y el rendimiento de diversas tecnologías de paralelización mediante la implementación del mismo algoritmo base en cada una de ellas.

2.2 Objetivos específicos

- Implementar la multiplicación de matrices en el framework asignado, partiendo del esqueleto provisto.
- Medir el tiempo de ejecución para los cuatro tamaños de entrada y calcular los GFLOPS efectivos.
- Comparar los resultados propios frente a las implementaciones de referencia (serial y OpenMP CPU) y frente a los demás grupos.
- Identificar las abstracciones clave del modelo de programación (cómo se expresa el paralelismo, cómo se gestiona la memoria, cómo se sincroniza).
- Presentar un análisis crítico de las ventajas, limitaciones y casos de uso del framework implementado.

3. Implementaciones de referencia

Las siguientes dos implementaciones serán presentadas por el profesor al inicio de la sesión y sirven como línea base para todos los grupos. Cada grupo debe incluir la comparación numérica con ambas en su exposición.

3.1 Serial (C++)

```
// matmul_serial.cpp - compilar: g++ -O2 -o serial matmul_serial.cpp
#include <cstdio>
#include <chrono>

void matmul_serial(const float* A, const float* B, float* C, int N) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            float acc = 0.0f;
            for (int k = 0; k < N; k++)
                acc += A[i*N + k] * B[k*N + j];
            C[i*N + j] = acc;
        }
}

int main() {
    const int N = 1024;
    float *A = new float[N*N], *B = new float[N*N], *C = new float[N*N];
    // TODO: inicializar A y B
    auto t0 = std::chrono::high_resolution_clock::now();
    matmul_serial(A, B, C, N);
    auto t1 = std::chrono::high_resolution_clock::now();
    double ms = std::chrono::duration<double, std::milli>(t1-t0).count();
    double gflops = 2.0*N*N*N / (ms*1e6);
    printf("Serial N=%d: %.2f ms %.2f GFLOPS\n", N, ms, gflops);
    delete[] A; delete[] B; delete[] C;
}
```

3.2 OpenMP CPU (referencia multicore)

```
// matmul_omp.cpp - compilar: g++ -O2 -fopenmp -o omp matmul_omp.cpp
#include <omp.h>
#include <cstdio>

void matmul_omp(const float* A, const float* B, float* C, int N) {
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            float acc = 0.0f;
            for (int k = 0; k < N; k++)
                acc += A[i*N + k] * B[k*N + j];
            C[i*N + j] = acc;
        }
}

int main() {
    const int N = 1024;
    // ... mismo harness de medición que en serial ...
    // Medir con omp_get_wtime() para mayor precisión en OpenMP
    double t0 = omp_get_wtime();
    matmul_omp(A, B, C, N);
    double ms = (omp_get_wtime() - t0) * 1000.0;
    printf("OpenMP CPU N=%d (hilos=%d): %.2f ms\n",
        N, omp_get_max_threads(), ms);
}
```

4. Asignación de grupos

Cada grupo es responsable de una implementación. La tabla resume la asignación, el hardware objetivo y el nombre convencional del ejecutable esperado:

Gru po	Implementación	Hardware	Ejecutable esperado
1	pthread (POSIX)	CPU multicore	matmul_pthreads
2	std::thread (C++11)	CPU multicore	matmul_stdthread
3	OpenMP GPU offload (target)	GPU / CPU fallback	matmul_omp_gpu
4	CUDA naive	GPU NVIDIA	matmul_cuda_naive
5	CUDA memoria compartida (tiled)	GPU NVIDIA	matmul_cuda_tiled
6	OpenCL	GPU / CPU / FPGA	matmul_opengl
7	SYCL / DPC++	GPU / CPU / FPGA	matmul_sycl

5. Descripción por grupo

Grupo 1: pthread (POSIX Threads)

Framework / API	pthread (POSIX Threads) — biblioteca del SO, disponible en Linux/macOS
Hardware objetivo	CPU multicore (sin GPU)
Requisitos previos	Conocimiento de mutex, join y particionamiento de trabajo

Esqueleto de código (punto de partida):

```
// matmul_pthreads.c — compilar: gcc -O2 -pthread -o matmul_pthreads
matmul_pthreads.c
#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>

#define N      1024
#define NTHREADS 8

float A[N*N], B[N*N], C[N*N];

typedef struct { int id; int nthreads; int n; } Args;

void* worker(void* arg) {
    Args* a = (Args*)arg;
    int chunk = a->n / a->nthreads;
    int inicio = a->id * chunk;
    int fin    = (a->id == a->nthreads-1) ? a->n : inicio + chunk;
    for (int i = inicio; i < fin; i++)
        for (int j = 0; j < a->n; j++) {
            float acc = 0.0f;
            for (int k = 0; k < a->n; k++)
                acc += A[i*a->n + k] * B[k*a->n + j];
            C[i*a->n + j] = acc;
        }
}
```

```

    }
    return NULL;
}

int main() {
    pthread_t hilos[NTHREADS];
    Args args[NTHREADS];
    // TODO: inicializar A y B, lanzar hilos, hacer join, medir tiempo
    for (int t = 0; t < NTHREADS; t++) {
        args[t] = (Args){t, NTHREADS, N};
        pthread_create(&hilos[t], NULL, worker, &args[t]);
    }
    for (int t = 0; t < NTHREADS; t++) pthread_join(hilos[t], NULL);
}

```

Aspectos a investigar y exponer:

- Overhead de creación de hilos vs. cantidad de hilos: ¿cuántos hilos son óptimos para su máquina?
- Estrategias de particionamiento: por filas, por columnas, por bloques 2D. ¿Cuál es más eficiente y por qué?
- Falsa compartición (false sharing): ¿cómo afecta al rendimiento cuando dos hilos escriben en la misma línea de caché?
- Escalabilidad: graficar GFLOPS vs. número de hilos (1, 2, 4, 8, 16). ¿Se observa la ley de Amdahl?
- Comparar con la referencia serial y OpenMP CPU.

Recursos sugeridos:

- man pthreads, POSIX Threads Programming — LLNL HPC Tutorials
- "Programming with POSIX Threads" — D. Butenhof (libro de referencia)
- perf stat / VTune para análisis de cache misses

Grupo 2: std::thread (C++11 / C++20)

Framework / API	std::thread, std::async, std::future — biblioteca estándar C++11 y posterior
Hardware objetivo	CPU multicore (sin GPU)
Requisitos previos	C++11: templates, lambdas, std::vector

Esqueleto de código (punto de partida):

```

// matmul_stdthread.cpp — compilar: g++ -O2 -std=c++17 -o matmul_stdthread
matmul_stdthread.cpp
#include <thread>
#include <vector>
#include <iostream>
#include <chrono>

void worker(const float* A, const float* B, float* C,
            int N, int inicio, int fin) {
    for (int i = inicio; i < fin; i++)
        for (int j = 0; j < N; j++) {
            float acc = 0.0f;
            for (int k = 0; k < N; k++)
                acc += A[i*N + k] * B[k*N + j];
            C[i*N + j] = acc;
        }
}

```

```
int main() {
    const int N = 1024;
    unsigned int T = std::thread::hardware_concurrency();
    std::vector<float> A(N*N), B(N*N), C(N*N);
    // TODO: inicializar A y B
    std::vector<std::thread> hilos;
    int chunk = N / T;
    for (unsigned int t = 0; t < T; t++) {
        int ini = t * chunk;
        int fin = (t == T-1) ? N : ini + chunk;
        hilos.emplace_back(worker, A.data(), B.data(), C.data(), N, ini, fin);
    }
    for (auto& h : hilos) h.join();
}
```

Aspectos a investigar y exponer:

- Comparación con pthreads: ¿qué abstrae std::thread? ¿Qué gana y qué pierde el programador?
- Uso de std::async y std::future para paralelismo de tareas (task parallelism) vs. paralelismo de datos.
- C++17 parallel algorithms (std::execution::par): demostrar std::transform con política de ejecución paralela.
- Thread pool manual vs. hilos por lanzamiento: impacto en el overhead cuando N es pequeño.
- Comparar con la referencia serial y OpenMP CPU.

Recursos sugeridos:

- cppreference.com — std::thread, std::async, std::execution
- "C++ Concurrency in Action" — Anthony Williams (2ª ed.)
- ISO C++17 §23.19 (Execution Policies)

Grupo 3: OpenMP GPU Offload (target)

Framework / API	OpenMP 4.5+ — directivas target, teams, distribute, parallel for
Hardware objetivo	GPU NVIDIA / AMD / Intel (o CPU si no hay GPU disponible)
Requisitos previos	OpenMP CPU (referencia), conceptos de jerarquía de memoria GPU

Esqueleto de código (punto de partida):

```
// matmul_omp_gpu.cpp — compilar:
// NVIDIA: nvc++ -O2 -mp=gpu -target=gpu -o matmul_omp_gpu matmul_omp_gpu.cpp
// AMD:    amdclang++ -O2 -fopenmp --offload-arch=gfx90a -o ...
// GCC fallback (sin GPU): g++ -O2 -fopenmp -o matmul_omp_gpu matmul_omp_gpu.cpp
#include <omp.h>
#include <cstdio>

void matmul_omp_gpu(const float* A, const float* B, float* C, int N) {
    #pragma omp target data map(to:A[0:N*N], B[0:N*N]) map(from:C[0:N*N])
    {
        #pragma omp target teams distribute parallel for collapse(2) \
            map(to:A[0:N*N],B[0:N*N]) map(from:C[0:N*N])
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++) {
                float acc = 0.0f;
                for (int k = 0; k < N; k++)
```

```

        acc += A[i*N + k] * B[k*N + j];
        C[i*N + j] = acc;
    }
}

int main() {
    const int N = 1024;
    float *A = new float[N*N], *B = new float[N*N], *C = new float[N*N];
    // TODO: inicializar, medir con omp_get_wtime()
    // Verificar con: omp_get_default_device() y omp_is_initial_device()
}

```

Aspectos a investigar y exponer:

- Cláusulas de mapeo: diferencia entre map(to:), map(from:), map(tofrom:) y map(alloc:). ¿Cuándo transferir datos y cuándo no?
- Jerarquía teams/distribute/parallel for: ¿cómo se mapea a grids/bloques en GPU? Comparar con CUDA/OpenCL.
- Cláusula num_teams y thread_limit: impacto en ocupancia de la GPU.
- Portabilidad real: demostrar compilación con GCC (sin GPU, como CPU offload) vs. nvc++ (con GPU NVIDIA).
- Comparar rendimiento con CUDA naive (Grupo 4): ¿cuánto cuesta la abstracción de portabilidad?

Recursos sugeridos:

- OpenMP 5.2 API Specification (openmp.org)
- "Using OpenMP — The Next Step" — Ruud van der Pas et al.
- NVIDIA HPC SDK documentation (nvc++ offloading)

Grupo 4: CUDA Naive

Framework / API	CUDA C/C++ — kernel con un hilo por elemento de C, acceso directo a memoria global
Hardware objetivo	GPU NVIDIA (Compute Capability ≥ 3.5)
Requisitos previos	Modelo de programación CUDA básico (grids, bloques, hilos), cudaMalloc/cudaMemcpy

Esqueleto de código (punto de partida):

```

// matmul_cuda_naive.cu — compilar: nvcc -O2 -arch=sm_86 -o matmul_cuda_naive
matmul_cuda_naive.cu
#include <cuda_runtime.h>
#include <cstdio>

__global__ void matmul_naive(const float* A, const float* B, float* C, int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int fila = blockIdx.y * blockDim.y + threadIdx.y;
    if (fila < N && col < N) {
        float acc = 0.0f;
        for (int k = 0; k < N; k++)
            acc += A[fila*N + k] * B[k*N + col];
        C[fila*N + col] = acc;
    }
}

int main() {

```

```

const int N = 1024, TILE = 16;
size_t bytes = N*N*sizeof(float);
float *h_A = new float[N*N], *h_B = new float[N*N], *h_C = new float[N*N];
float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, bytes); cudaMalloc(&d_B, bytes); cudaMalloc(&d_C, bytes);
// TODO: inicializar h_A, h_B y copiar al device
dim3 bloque(TILE, TILE);
dim3 grid((N+TILE-1)/TILE, (N+TILE-1)/TILE);
// Medir con cudaEvent
cudaEvent_t t0, t1;
cudaEventCreate(&t0); cudaEventCreate(&t1);
cudaEventRecord(t0);
matmul_naive<<<grid, bloque>>>(d_A, d_B, d_C, N);
cudaEventRecord(t1); cudaEventSynchronize(t1);
float ms; cudaEventElapsedTime(&ms, t0, t1);
double gflops = 2.0*N*N*N / (ms*1e6);
printf("CUDA naive N=%d: %.2f ms %.2f GFLOPS\n", N, ms, gflops);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}

```

Aspectos a investigar y exponer:

- Modelo de ejecución CUDA: warp, SM, ocupancia. ¿Cuántos warps activos hay por SM en su configuración?
- Patrón de acceso a memoria global: coalescencia en A (filas) vs. no coalescencia en B (columnas con stride N). Medir con Nsight Compute.
- Intensidad aritmética (FLOP/byte): calcular teóricamente y comparar con el modelo roofline de la GPU.
- Variar el tamaño del bloque (8×8, 16×16, 32×32): ¿cómo cambia el rendimiento? ¿Por qué?
- Comparar con referencia serial, OpenMP CPU, y especialmente con Grupo 5 (CUDA tiled) para cuantificar el beneficio de la memoria compartida.

Recursos sugeridos:

- CUDA C++ Programming Guide (docs.nvidia.com)
- Nsight Compute — analizador de kernels CUDA
- "Programming Massively Parallel Processors" — Kirk & Hwu (4ª ed.)

Grupo 5: CUDA con Memoria Compartida (Tiled)

Framework / API	CUDA C/C++ — kernel tiled con <code>__shared__</code> memory y sincronización <code>__syncthreads()</code>
Hardware objetivo	GPU NVIDIA (Compute Capability ≥ 3.5)
Requisitos previos	CUDA naive (Grupo 4), jerarquía de memoria en GPU, concepto de tile

Esqueleto de código (punto de partida):

```

// matmul_cuda_tiled.cu — compilar: nvcc -O2 -arch=sm_86 -o matmul_cuda_tiled
matmul_cuda_tiled.cu
#define TILE 16

__global__ void matmul_tiled(const float* A, const float* B, float* C, int N) {
    __shared__ float tA[TILE][TILE];
    __shared__ float tB[TILE][TILE];
    int tx = threadIdx.x, ty = threadIdx.y;
    int col = blockIdx.x*TILE + tx;

```

```

int fila = blockIdx.y*TILE + ty;
float acc = 0.0f;
for (int t = 0; t < (N+TILE-1)/TILE; t++) {
    int idxAk = t*TILE + tx, idxBk = t*TILE + ty;
    tA[ty][tx] = (fila < N && idxAk < N) ? A[fila*N + idxAk] : 0.0f;
    tB[ty][tx] = (idxBk < N && col < N) ? B[idxBk*N + col] : 0.0f;
    __syncthreads();
    for (int k = 0; k < TILE; k++) acc += tA[ty][k] * tB[k][tx];
    __syncthreads();
}
if (fila < N && col < N) C[fila*N + col] = acc;
}

// main: mismo harness de medición que en Grupo 4

```

Aspectos a investigar y exponer:

- Reducción de accesos a memoria global: calcular teóricamente el factor TILE y verificar con Nsight Compute (L2 hit rate, global load transactions).
- El rol de `__syncthreads()`: demostrar con un ejemplo qué ocurre si se elimina la primera o la segunda barrera.
- Intensidad aritmética mejorada: comparar el valor teórico con el del naive. ¿Se llega al ridge point del roofline?
- Experimento con TILE=8, 16, 32: trade-off entre tamaño de tile, uso de memoria compartida por SM y ocupancia.
- Comparar con Grupo 4 (CUDA naive), Grupo 6 (OpenCL) y Grupo 7 (SYCL). ¿Cuánto influye el conocimiento del hardware en el rendimiento?

Recursos sugeridos:

- "Programming Massively Parallel Processors" — Kirk & Hwu, Cap. 4–5
- CUDA Best Practices Guide — sección Shared Memory
- Nsight Compute Roofline Analysis

Grupo 6: OpenCL

Framework / API	OpenCL 1.2 / 2.0 — estándar abierto, portátil entre GPU NVIDIA, AMD, Intel y FPGAs
Hardware objetivo	GPU NVIDIA, AMD, Intel, CPU (cualquier dispositivo OpenCL disponible)
Requisitos previos	Conceptos básicos de CUDA o GPU computing, C/C++

Esqueleto de código (punto de partida):

```

// matmul_openc1.cpp — compilar: g++ -O2 -o matmul_openc1 matmul_openc1.cpp
// -lOpenCL
// Kernel OpenCL (string inline o archivo .cl)
const char* kernel_src = R"(
__kernel void matmul(__global const float* A,
                    __global const float* B,
                    __global float* C, int N) {
    int col = get_global_id(0);
    int fila = get_global_id(1);
    if (fila < N && col < N) {
        float acc = 0.0f;
        for (int k = 0; k < N; k++)
            acc += A[fila*N + k] * B[k*N + col];
        C[fila*N + col] = acc;
    }
}
)";

```



```

    }
}
)"
// Host (boilerplate OpenCL):
// 1. clGetPlatformIDs / clGetDeviceIDs
// 2. clCreateContext + clCreateCommandQueue
// 3. clCreateBuffer (A, B, C)
// 4. clCreateProgramWithSource + clBuildProgram
// 5. clCreateKernel + clSetKernelArg
// 6. clEnqueueWriteBuffer (host → device)
// 7. clEnqueueNDRangeKernel con global={N,N}, local={TILE,TILE}
// 8. clEnqueueReadBuffer (device → host)
// 9. Medir con cl_event y clGetEventProfilingInfo

```

Aspectos a investigar y exponer:

- Arquitectura del runtime OpenCL: plataforma, contexto, cola de comandos, buffer, kernel. ¿Por qué hay más boilerplate que en CUDA?
- Portabilidad real: ejecutar el mismo binario en GPU y en CPU. ¿Cambia el rendimiento? ¿Por qué?
- Extensión a versión con memoria local (`__local`, equivalente a `__shared__` de CUDA): implementar el tiling y medir la ganancia.
- Comparar el modelo de programación con CUDA: work-item ↔ thread, work-group ↔ bloque, NDRange ↔ grid.
- Comparar rendimiento con Grupo 4 (CUDA naive) en la misma GPU y con Grupo 7 (SYCL).

Recursos sugeridos:

- OpenCL 3.0 Reference Pages ([khronos.org/opencl](https://www.khronos.org/opencl))
- "OpenCL Programming Guide" — Munshi, Gaster et al.
- `clinfo` — herramienta para inspeccionar plataformas y dispositivos disponibles

Grupo 7: SYCL / DPC++

Framework / API	SYCL 2020 (oneAPI DPC++) — C++ moderno, sin macros especiales, portátil entre GPUs Intel/NVIDIA/AMD
Hardware objetivo	GPU Intel (oneAPI), GPU NVIDIA con SYCL-CUDA backend, CPU
Requisitos previos	C++17 (lambdas, plantillas), conceptos básicos de GPU computing

Esqueleto de código (punto de partida):

```

// matmul_sycl.cpp — compilar (Intel oneAPI):
// icpx -O2 -fsycl -o matmul_sycl matmul_sycl.cpp
// compilar (AdaptiveCpp / hipSYCL para NVIDIA/AMD):
// acpp --acpp-targets="cuda:sm_86" -O2 -o matmul_sycl matmul_sycl.cpp
#include <sycl/sycl.hpp>
#include <vector>
#include <iostream>

int main() {
    const int N = 1024;
    sycl::queue q{ sycl::default_selector_v };
    std::cout << "Dispositivo: "
                << q.get_device().get_info<sycl::info::device::name>() << "\n";

    std::vector<float> h_A(N*N,1.f), h_B(N*N,1.f), h_C(N*N,0.f);

```

```

sycl::buffer<float,2> A(h_A.data(), sycl::range<2>{(size_t)N,(size_t)N});
sycl::buffer<float,2> B(h_B.data(), sycl::range<2>{(size_t)N,(size_t)N});
sycl::buffer<float,2> C(h_C.data(), sycl::range<2>{(size_t)N,(size_t)N});

q.submit([&](sycl::handler& h) {
    auto a = A.get_access<sycl::access::mode::read>(h);
    auto b = B.get_access<sycl::access::mode::read>(h);
    auto c = C.get_access<sycl::access::mode::write>(h);
    h.parallel_for(sycl::range<2>{(size_t)N,(size_t)N},
        [=](sycl::id<2> idx) {
            int fila = idx[0], col = idx[1];
            float acc = 0.0f;
            for (int k = 0; k < N; k++)
                acc += a[fila][k] * b[k][col];
            c[fila][col] = acc;
        });
    }).wait();
// TODO: medir con std::chrono o sycl::event::get_profiling_info
}

```

Aspectos a investigar y exponer:

- Modelo de programación SYCL: queue, buffer/accessor vs. USM (Unified Shared Memory). ¿Cuándo conviene cada modelo de memoria?
- Ventaja sobre CUDA/OpenCL: C++ estándar sin macros, un solo binario para múltiples backends (CUDA, HIP, OpenCL, Level Zero).
- Versión con memoria local (sycl::local_accessor): equivalente al tiling de CUDA pero expresado en C++ moderno.
- Ecosistema actual: oneAPI (Intel), AdaptiveCpp/hipSYCL, triSYCL. ¿Cuál elegir y por qué?
- Comparar rendimiento con Grupo 6 (OpenCL) y Grupo 4/5 (CUDA) en el mismo hardware. ¿Cuánto cuesta la abstracción de portabilidad?

Recursos sugeridos:

- SYCL 2020 Specification (khronos.org/sycl)
- "Data Parallel C++" — Reinders, Ashbaugh et al. (Intel Press, acceso abierto en github.com/Apress/data-parallel-CPP)
- Intel oneAPI Toolkit (free download — funciona en CPU sin GPU Intel)
- AdaptiveCpp (antes hipSYCL) — backend CUDA para SYCL en GPU NVIDIA

6. Formato de la exposición

Duración: 10 a 12 minutos de exposición + 3 minutos de preguntas por parte del resto del grupo y del profesor.

Estructura sugerida: (1) Introducción al framework (2 min) · (2) Demo de compilación y ejecución en vivo o grabada (3 min) · (3) Resultados de rendimiento comparativo (3 min) · (4) Análisis y conclusiones (3 min).

Entregables: Código fuente en repositorio o archivo comprimido, y diapositivas o documento de resultados en formato PDF.

Fecha de entrega del código: 72 horas antes de la sesión de exposiciones para revisión previa.

Orden de exposición: CPU threads primero (Grupos 1 y 2), luego GPU (Grupos 3–7). El profesor presenta las referencias al inicio de la sesión.

7. Harness de medición y verificación

Para facilitar la comparación entre grupos, se recomienda usar el siguiente protocolo de medición:

```
// Protocolo de medición común (adaptar a cada framework)

// 1. Inicialización determinista (para verificación)
//   A[i][j] = (float)(i + j) / N;
//   B[i][j] = (float)(i - j + N) / N;

// 2. Corrida de calentamiento (warm-up): 1 ejecución sin medir

// 3. Medición: NREP = 5 repeticiones, reportar la mediana
#define NREP 5

// 4. Cálculo de GFLOPS:
//   double gflops = 2.0 * (double)N * N * N / (t_ms * 1e6);

// 5. Verificación numérica (solo para N <= 1024):
//   Comparar C[i][j] con resultado serial usando tolerancia abs < 1e-3
//   Reportar el error máximo absoluto

// 6. Tabla de resultados esperada:
//   N      | t (ms) | GFLOPS | Speedup vs serial | Speedup vs OMP CPU
//   512    |        |        |                   |
//   1024   |        |        |                   |
//   2048   |        |        |                   |
//   4096   |        |        |                   |
```

8. Tabla resumen de frameworks

Esta tabla será completada colectivamente al final de la sesión de exposiciones, con los valores medidos por cada grupo:

Framework	Lenguaje / Estándar	Hardware	Portabilidad	GFLOPS (N=2048)
Serial C++	C++17	CPU (1 núcleo)	★★★★★	—
OpenMP CPU	C/C++, directivas	CPU multinúcleo	★★★★★	—
pthread (G1)	C (POSIX)	CPU multinúcleo	★★★★☆	
std::thread (G2)	C++11/17/20	CPU multinúcleo	★★★★★	
OMP GPU offload (G3)	C/C++, directivas	GPU / CPU	★★★★☆	
CUDA naive (G4)	CUDA C/C++	GPU NVIDIA	★★☆☆☆	
CUDA tiled (G5)	CUDA C/C++	GPU NVIDIA	★★☆☆☆	
OpenCL (G6)	OpenCL C / C++	GPU/CPU/ FPGA	★★★★★	

Framework	Lenguaje / Estándar	Hardware	Portabilidad	GFLOPS (N=2048)
SYCL / DPC++ (G7)	C++17 + SYCL	GPU/CPU/ FPGA	★★★★★	

★ Portabilidad: se califica según soporte en NVIDIA / AMD / Intel / CPU / FPGA sin recompilación.